

Class Overview

In Class:

- Integration Testing
- System Testing
- Metamorphic Testing

After Class Office Hours: Homework Help

Homework:

- 04-01: Mutation Testing
- 04-08: Metamorphic Testing

Clicker Points (Current): 3

Note: Review volume tends to be high towards the end of the semester!

1

Tentative Remaining Schedule

Week	Plan
04/01 (Today)	Integration, System, Metamorphic Testing
04/08	Reviews, Model Checking
04/15	Search-Based Testing, Exploratory Testing, Final Exam Review
04/22*	Final Assessment, Half-Semester Project Due
04/29	Half-Semester Project Presentations
* No extensions past this point	

2

CIS 613: Integration Testing

3

The Mars Climate Orbiter Mission

- Mission failed in September 1999
- Completed successful flight of 416,000,000 miles (665.600.600 km)
- 41 weeks flight duration
- Lost at beginning of Mars orbit
- An **integration fault**: Lockheed Martin used English units for acceleration calculations (pounds), and Jet Propulsion Laboratory used metric units (newtons).
- NASA announced a US\$ 50,000 project to discover how this happened.

4

Goals/Purpose of Integration Testing

- Presumes previously tested units
- Not system testing
- Tests functionality "between" unit and system levels
- Basis for test case identification?
- Emphasis shifts from "how to test" to "what to test" (Model-Based Testing)

5

Testing Level Assumptions and Objectives

	Assumptions	Goals
Unit	• All other units are correct • Compiles correctly	• Correct unit function • Coverage metrics satisfied
Integration	• Unit testing complete	• Interfaces correct • Correct function across units • Fault isolation support
System	• Integration testing complete • Tests occur at port boundary	• Correct system functions • Non-functional requirements tested • Customer satisfaction

6

Approaches to Integration Testing

("source" of test cases)

- Functional Decomposition (most common)
 - Top-down
 - Bottom-up
 - Sandwich
 - "Big bang"
- Call graph
 - Pairwise integration
 - Neighborhood integration
- Paths
 - MM-Paths

7

Basis of Integration Testing Strategies

- **Functional Decomposition:** applies best to procedural code
- **Call Graph:** applies to both procedural and object-oriented code
- **MM-Paths:** applies to both procedural and object-oriented code

8

Calendar Example

- Date in the form mm, dd, yyyy
- Calendar functions:
 - the date of the **next day** (i.e., NextDate)
 - the **day of the week** corresponding to the date
 - the **zodiac sign** of the date
 - the most **recent year in which Memorial Day was celebrated on May 27**
 - the most recent **Friday the Thirteenth**

9

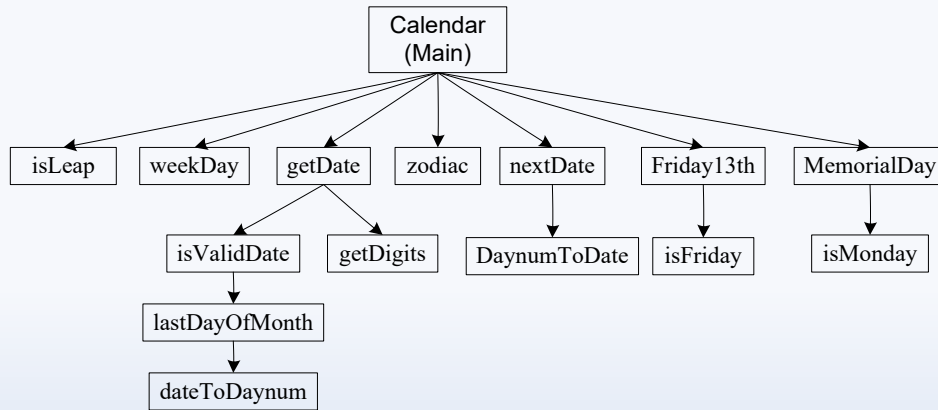
Calendar Program Units

```

Main  Calendar
      Function isLeap
      Procedure weekDay
      Procedure getDate
          Function isValidDate
              Function lastDayOfMonth
          Procedure getDigits
      Procedure memorialDay
          Function isMonday
      Procedure friday13th
          Function isFriday
      Procedure nextDate
          Procedure dayNumToDate
      Procedure zodiac
  
```

10

Calendar Program Decomposed



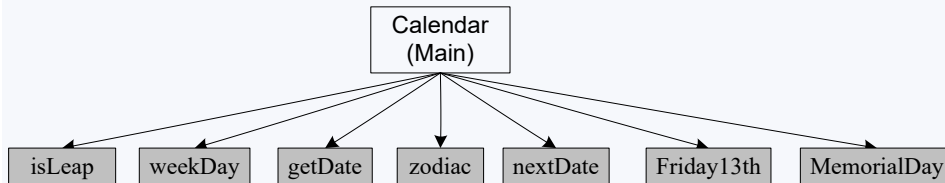
11

Top-Down Integration Mechanism

- Breadth-first traversal of the functional decomposition tree.
- First step: Check main program logic, with all called units replaced by stubs that always return correct values.
- Move down one level
 - replace one stub at a time with actual code.
 - any fault must be in the newly integrated unit

12

First Step in Top-Down Integration



“Grey” units are stubs that return the correct values when referenced. This level checks the main program logic.

13

weekDayStub

Procedure weekDayStub(mm, dd, yyyy, dayName)

If ((mm = 10) AND (dd = 28) AND (yyyy = 2013))

Then dayName = “Monday”

EndIf

.

.

.

If ((mm = 10) AND (dd = 30) AND (yyyy = 2013))

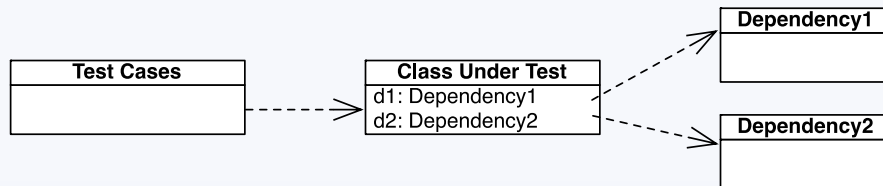
Then dayName = “Wednesday”

EndIf

14

Unit Testing

Test Cases has a dependency to the **Class Under Test**, in order to be able to test it. The **Class Under Test** also has dependencies that must be included in the test in order for the **Class Under Test** to work.

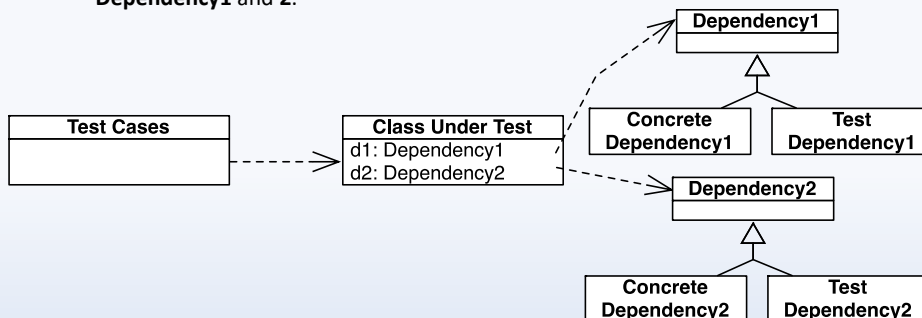


Would there be any problems with this?

15

Unit Testing

Class Under Test has a dependency to **Dependency1** and **Dependency2**, which are interfaces. The *real* code for the program is represented by **Concrete Dependency1** and **2**, while test code is *stubbed* into **Test Dependency1** and **2**.



16

JUnit Test Case for Leap Year and Non-Leap Year

```

1 import org.junit.Test;
2 import static org.junit.Assert.*;

3 public class SimpleDateTest {

4     @Test
5     public void testWithDependency() {
6         // This test has a dependency on SimpleDate.isLeap
7         SimpleDate simpleDate = new SimpleDate(1, 1, 2000);

8         assertTrue(simpleDate.rangesOK(2, 29, 2000)); // Valid due to leap year
9         assertFalse(simpleDate.rangesOK(2, 29, 2001)); // Invalid due to leap year
10    }
11 }

```

17

Manual Dependency Removal

```

1 @Test
2 public void testWithoutDependencyManual() {
3     // This test has no dependency on SimpleDate.isLeap
4     SimpleDate simpleDate = new SimpleDate(1, 1, 2000) {
5         @Override
6         protected boolean isLeap(int year) {
7             if(2000 == year)
8                 return true;
9             else if(2001 == year)
10                return false;
11             else
12                throw new IllegalArgumentException("No Mock for year " + year);
13         }
14     };

15     assertTrue(simpleDate.rangesOK(2, 29, 2000)); // Valid due to leap year
16     assertFalse(simpleDate.rangesOK(2, 29, 2001)); // Valid due to leap year
17 }

```

18

Automatic Dependency Removal Using Mockito

```
1  @Test
2  public void testWithoutDependencyAutomatic() {
3      // This test removes a dependency on SimpleDate.isLeap
4      SimpleDate simpleDate = mock(SimpleDate.class, CALLS_REAL_METHODS);
5
6      when(simpleDate.isLeap(2000)).thenReturn(true);
7      when(simpleDate.isLeap(2001)).thenReturn(false);
8
9      assertTrue(simpleDate.rangesOK(2, 29, 2000)); // Valid due to leap year
10     assertFalse(simpleDate.rangesOK(2, 29, 2001)); // Invalid due to leap year
11 }
```

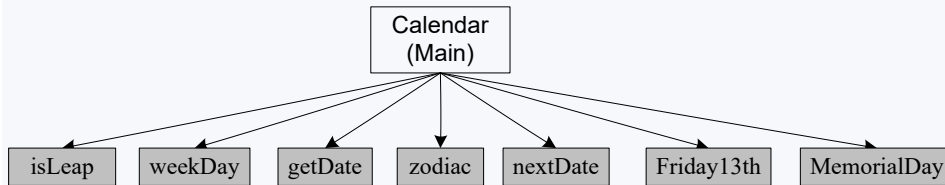
19

The Best Part?

Mocking Tools Exist for Most Languages

20

First Step in Top-Down Integration

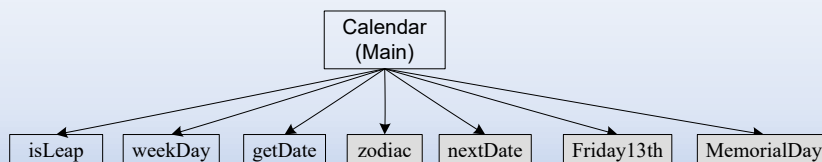
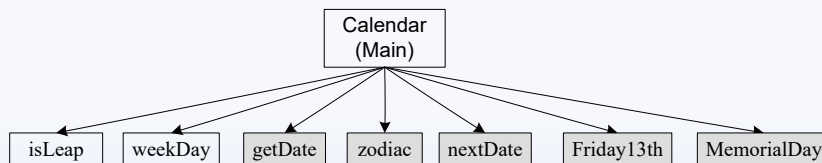
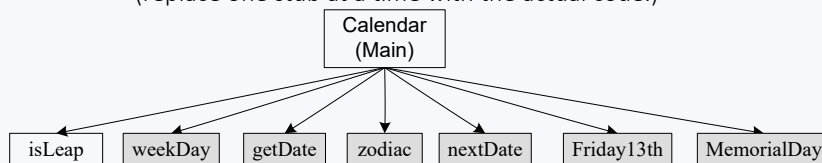


"Grey" units are stubs that return the correct values when referenced. This level checks the main program logic.

21

Next Three Steps

(replace one stub at a time with the actual code.)



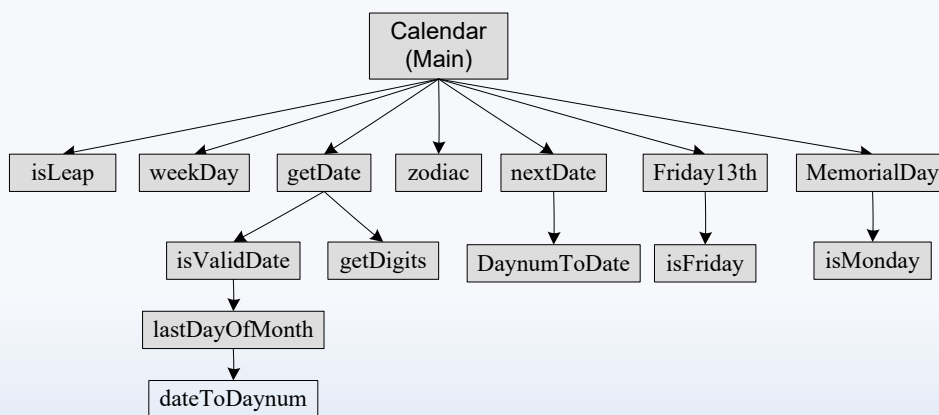
22

Bottom-Up Integration Mechanism

- Reverse of top-down integration
- Start at leaves of the functional decomposition tree.
- Driver units...
 - call next level unit
 - serve as a small test bed
 - “drive” the unit with inputs
 - drivers know expected outputs
- As with top-down integration, one driver unit at a time is replaced with actual code.
- Any fault is (most likely) in the newly integrated code.

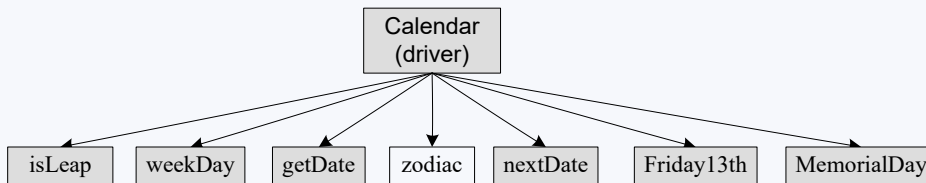
23

Starting Point of Bottom-Up Integration



24

Bottom-Up Integration of Zodiac



25

Top-Down and Bottom-Up Integration

- Both depend on throwaway code.
 - drivers are usually more complex than stubs
- Both test just the interface between two units at a time.
- In Bottom-Up integration, a driver might simply reuse unit level tests for the "lower" unit.
- Fan-in and fan-out in the decomposition tree results in some redundancy.

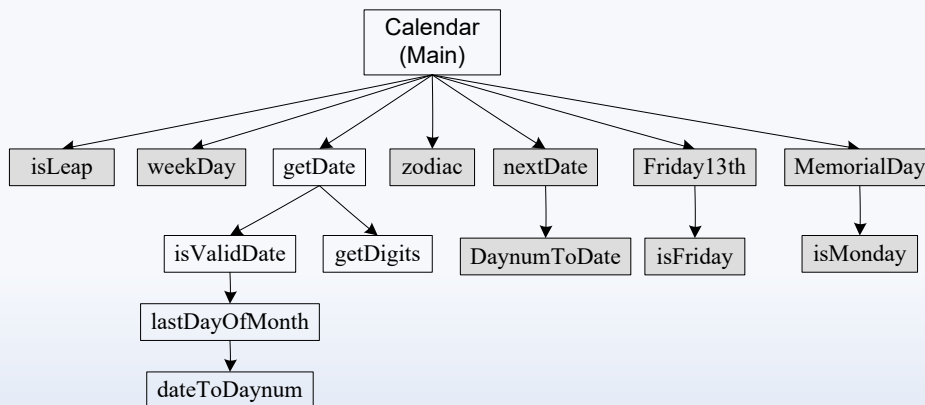
26

Sandwich Integration

- Avoids some of the repetition on both top-down and bottom-up integration.
- Nicely understood as a depth-first traversal of the functional decomposition tree.
- A “sandwich” is one path from the root to a leaf of the functional decomposition tree.
- Avoids stub and driver development.
- More complex fault isolation.

27

Real Sandwich Example



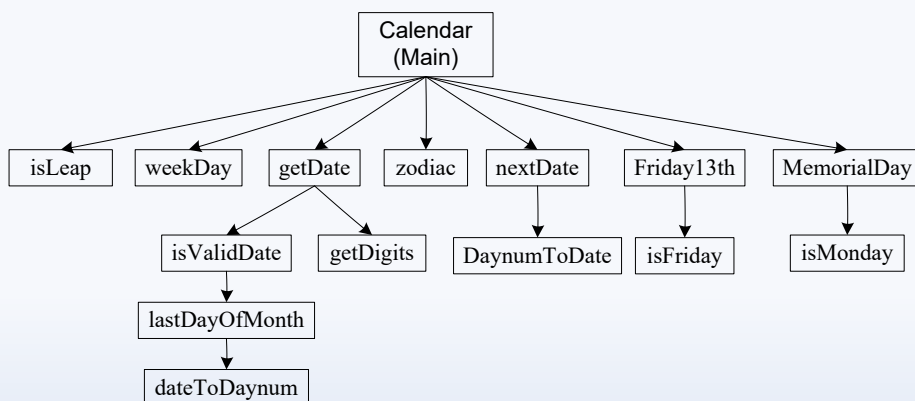
28

“Big Bang” Integration

- No...
 - stubs
 - drivers
 - strategy
- And very difficult fault isolation
- (Named after one of the theories of the origin of the Universe)

29

“Big Bang” Integration



30

Pros and Cons of Decomposition-Based Integration

- Pros
 - intuitively clear
 - “build” with proven components
 - fault isolation varies with the number of units being integrated
- Cons
 - some branches in a functional decomposition may not correspond with actual interfaces.
 - stub and driver development can be extensive

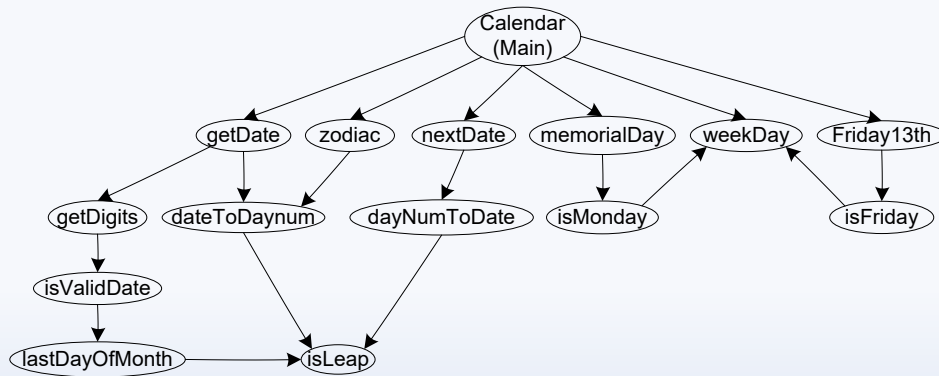
31

Call Graph-Based Integration

- Definition: The Call Graph of a program is a directed graph in which
 - nodes are unit
 - edges correspond to actual program calls (or messages)
- Call Graph Integration avoids the possibility of impossible edges in decomposition-based integration.
- Can still use the notions of stubs and drivers.
- Can still traverse the Call Graph in a top-down or bottom-up strategy.

32

Call Graph of the Calendar Program



33

Call Graph-Based Integration Strategies

- Two strategies
 - Pair-wise integration
 - Neighborhood integration
- Degrees of nodes in the Call Graph indicate integration sessions
 - isLeap and weekDay are each used by three units
- Possible priorities for testing
 - test high indegree nodes first, or at least,
 - pay special attention to “popular” nodes

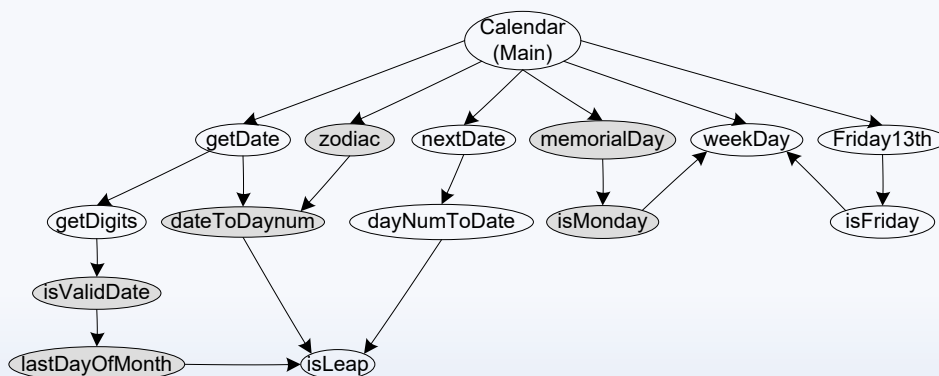
34

Pair-Wise Integration

- By definition, and edge in the Call Graph refers to an interface between the units that are the endpoints of the edge.
- Every edge represents a pair of units to test.
- Still might need stubs and drivers
- Fault isolation is localized to the pair being integrated

35

Example pairs for Pair-Wise Integration



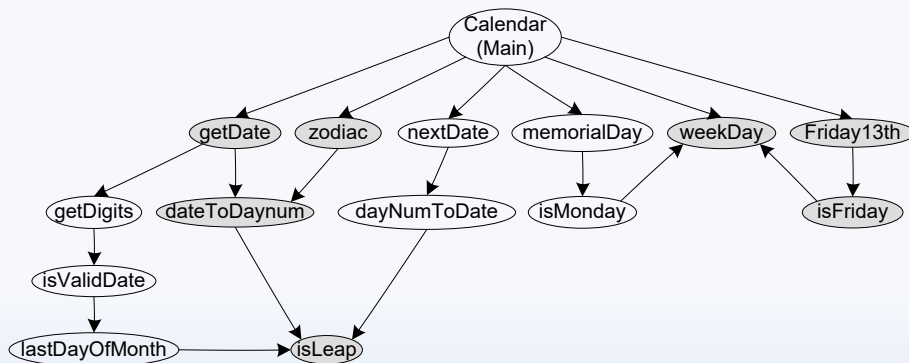
36

Neighborhood Integration

- The neighborhood (or radius) of a node in a graph is the set of nodes that are one edge away from the given node.
- This can be extended to larger sets by choosing larger values for the radius.
- Stub and driver effort is reduced.

37

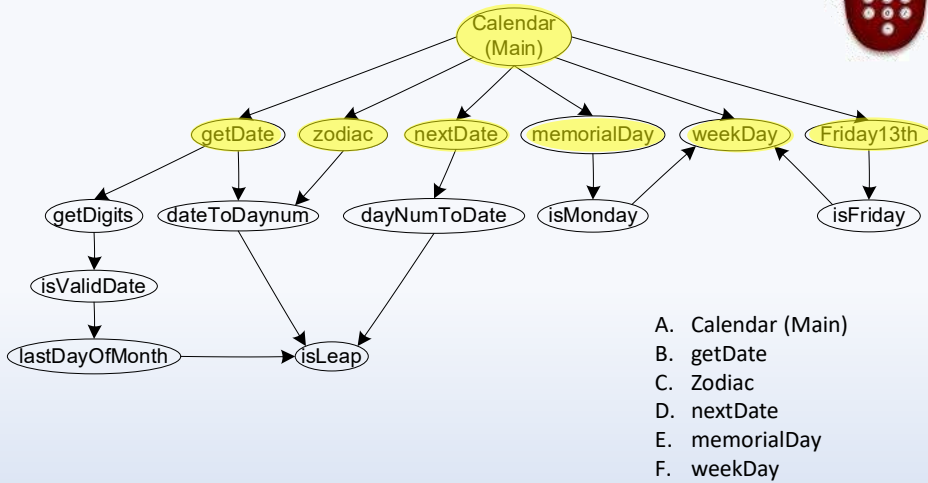
Example neighborhoods for Neighborhood Integration



What is the neighborhood radius for the neighborhoods?

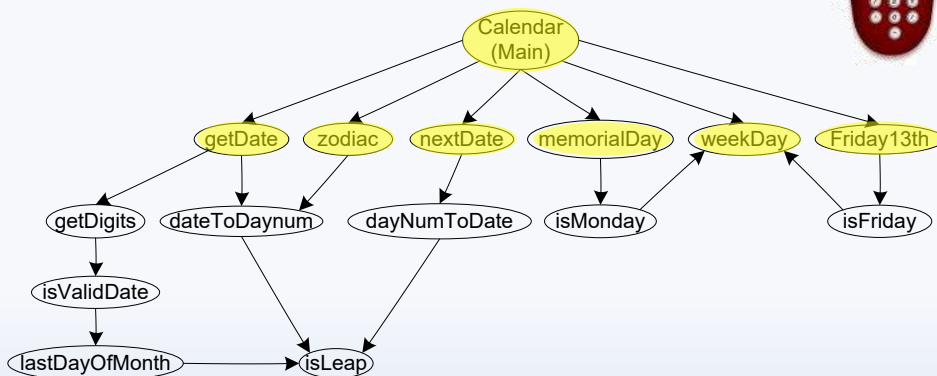
38

What is the center of the neighborhood?



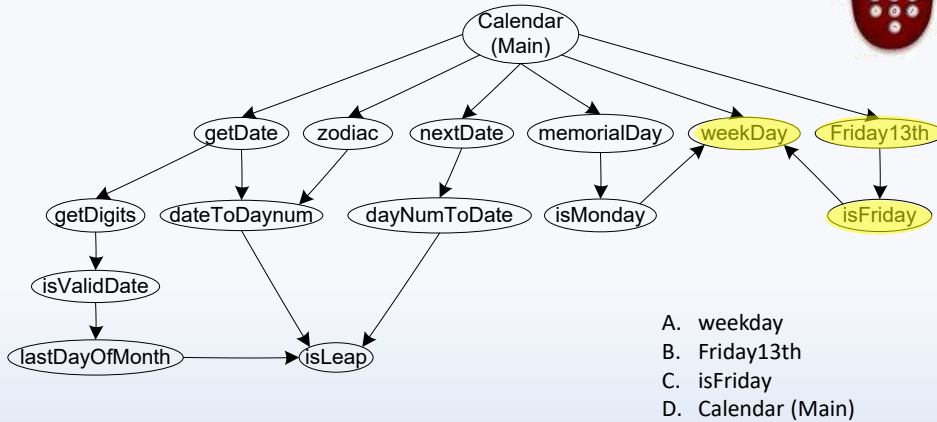
39

What is the neighborhood?



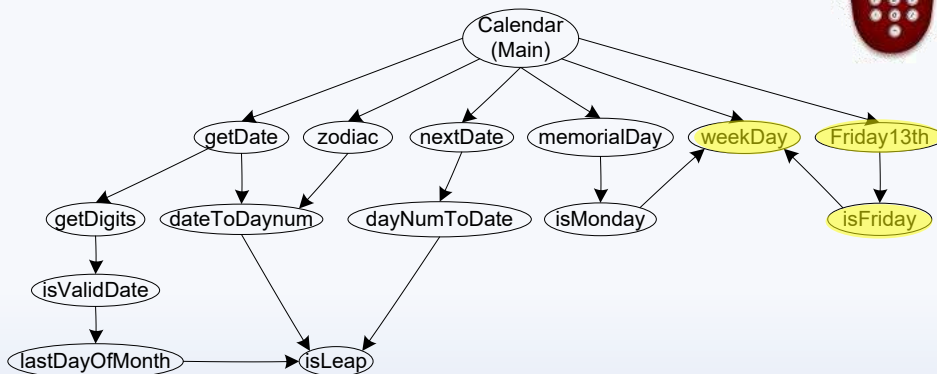
40

What is the center of the neighborhood?



41

What is the neighborhood?



42

Path-Based Integration

- An integration testing level construct similar to DD-Paths for unit testing, except for integration
 - extend the symbiosis of spec-based and code-based testing to the integration level
 - greater emphasis on behavioral threads
 - shift emphasis from interface testing to interactions (co-functions) among units
- New definitions needed
 - source and sink nodes in a program graph
 - module (unit) execution path
 - generalized message
 - MM-Path

43

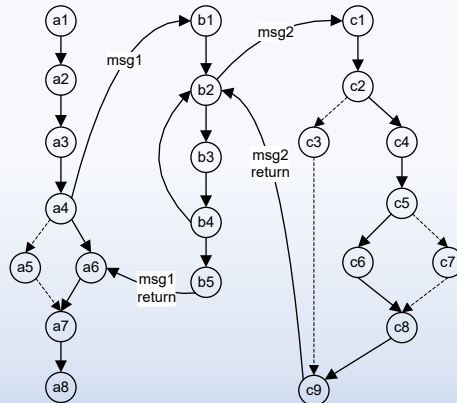
New and Extended Definitions

- A **source node** in a program is a statement fragment at which program execution begins or resumes.
- A **sink node** in a unit is a statement fragment at which program execution terminates.
- A **module execution path** is a sequence of statements that begins with a source node and ends with a sink node, with no intervening sink nodes.
- A **message** is a programming language mechanism by which one unit transfers control to another unit, and acquires a response from the other unit.

44

MM-Path Definition & Example

- An MM-Path is an interleaved sequence of module execution paths and messages.
- An MM-Path across three units:



**Could we
compute
cyclomatic
complexity of
MM-Paths?**

45

Comparison of Integration Testing Strategies

	Ability to test interfaces	Ability to test co-functionality	Fault isolation and resolution
Functional Decomposition	acceptable, but can be deceptive (phantom edges)	limited to pairs of units	Identifies Unit
Call Graph	acceptable	limited to pairs of units	Identifies Unit
MM-Path	excellent	complete	Identifies Unit & Path

46